

```
from memory_profiler_sample import measure_ram #Note: imported
function is from above sample
from array import array
# Create many integers
N = 5_000_000
```

```
@measure_ram
def sample_list():
    print("List Creation")
    data_list = list(range(N))
```

```
sample_list()
```

```
@measure_ram
def sample_arr():
    print("Array Creation")
    data_array = array('i', range(N))
```

```
sample_arr()
```

```
##OUTPUT
% /usr/bin/python3 list_VS_arr_sample.py
List Creation
RAM used: 41.45 MB
Array Creation
RAM used: 19.16 MB
```

Tuple vs list: Tuples use less memory as they cannot be resized and no extra capacity is reserved. They are faster and more cache-friendly. The same is true of set vs dict.

Choose the smallest numeric type: When working with large datasets in Python, NumPy arrays are far more memory-efficient than pure Python lists or Pandas Series (which often default to higher-precision types). The key is to select the smallest dtype that can safely represent your data without overflow or loss of precision. Here are a few options: int64 uses 8 bytes, int32 uses 4 bytes, int16 uses 2 bytes.

Use sparse data structures: Use libraries like scipy.sparse or pandas.SparseArray when the data holds most values as zero -- storing all zeros does not make effective use of RAM. Hence sparse matrices store only non-zero entries.

Bytes vs Unicode

Python distinguishes between text (human-readable characters) and binary data (raw bytes — images, files, network packets, compressed data, encrypted data). Understanding the difference is essential for file handling, networking, APIs, and encoding issues. Unicode strings use more memory because character metadata needs to be stored. Bytes are compact as they use only 1 byte per element.

```
import sys
```

```
def get_object_size_mb(obj):
    """
    Returns the size of a Python object in megabytes (MB).
    """
    size_bytes = sys.getsizeof(obj)
    size_mb = size_bytes / (1024 * 1024) # Convert bytes to
    MB
    return size_mb
```

```
# Create N string
N = 500_000_000
```

```
s = "hello "
```

```
# Unicode string (each character may take multiple bytes)
def sample_text():
    print("Text Creation")
    data_text = s*N
    print(f"Size is: {get_object_size_mb(data_text):.2f}
    MB")
```

```
sample_text()
```

```
# Bytes version (UTF-8 encoded)
def sample_bytes():
    print("Bytes Creation")
    data_text = s*N
    data_bytes = data_text.encode("utf-8")
    print(f"Size is: {get_object_size_mb(data_bytes):.2f}
    MB")
```

```
sample_bytes()
```

```
##OUTPUT
% /usr/bin/python3 text_VS_unicode_sample.py
Text Creation
Size is: 13351.44 MB
Bytes Creation
Size is: 4768.37 MB
```

Handling large text data stores

Handling massive text data (logs, documents, transcripts, crawled data, chat history, etc) requires storage formats and structures that are memory-efficient, compressed, and scalable.

Compression (gzip, bz2, zstd): Use of compression reduces the size of the text by 10-30X -- storing raw text wastes huge amounts of space.

mmap: Map a file (or part of it) directly into memory and treat gigabytes of data as a giant string or byte array. This is